

FULL STACK WEB DEVELOPMENT - SOURCE CODE

Student Name: Kottana Roshini

Roll No.: A24126552263

Question 2: Node.js File Management

File: fileManager.js

```
// fileManager.js
// Node.js File Management using the built-in File System (fs) module.
// Operations: create/write a file, read it, append content, display final text.

// Import the built-in modules (no installation needed)
const fs = require("fs");           // File System: create, read, append files
const path = require("path");       // Path: work safely with file names
const readline = require("readline"); // Readline: take input from the terminal

// Create an interface to read what the user types in the terminal
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Lets us read the typed lines one by one using "await"
const lines = rl[Symbol.asyncIterator]();

// Helper: show a question and wait for the user's answer
async function ask(question) {
  process.stdout.write(question);
  const { value } = await lines.next();
  return value === undefined ? "" : value;
}

// Helper: read a file and display its contents on the screen
function displayFile(filePath, heading) {
  // Read the whole file as text (utf8)
  const data = fs.readFileSync(filePath, "utf8");
  console.log("\n--- " + heading + " ---");
  console.log(data);
  console.log("-----");
}

async function main() {
  // try...catch handles any file error (for example, permission problems)
  try {
    // 1. Accept the file name and content from the user
    let fileName = (await ask("Enter file name: ")).trim();
    if (fileName === "") {
      // Validation: an empty name is not allowed
      console.log("Error: File name cannot be empty.");
      return;
    }
    fileName = path.basename(fileName); // avoid folder paths
    if (path.extname(fileName) === "") {
      fileName += ".txt"; // default extension
    }
    // Full path: the file is created in the same folder as this program
    const filePath = path.join(__dirname, fileName);
    const content = await ask("Enter file content: ");

    // 2. Create / write the file (an existing file is overwritten)
    const existed = fs.existsSync(filePath);
    fs.writeFileSync(filePath, content);
    const status = existed ? "Existing file overwritten: " : "File created: ";
    console.log("\n" + status + fileName);

    // 3. Read the contents
    displayFile(filePath, "File contents after writing");

    // 4. Append additional content at the end of the file
    const extra = await ask("Enter additional content to append: ");
    fs.appendFileSync(filePath, "\n" + extra);
    console.log("\nAdditional content appended successfully.");

    // 5. Display the final contents
  }
}
```

```
        displayFile(filePath, "Final file contents");
    } catch (err) {
        // Runs if any operation above fails
        console.log("File operation failed: " + err.message);
    } finally {
        // Always close the readline interface so the program can exit
        rl.close();
    }
}

// Start the program
main();
```